Session 7 Assessment

State Deep Dive, Input Validation, UI Feedback and Comprehensive Feature Design

Submitted By: Ruchi Kumari
Course: Android App Development Using GenAI
Internship: MindMatrix - Android App Development Using Gen AI
Session: Session 7 - State Concepts, Input and Validation, Feature Design

Assessment Overview

Task	Task Title	Focus Areas
Task 1	State Concepts Deep Dive and Visual Mapping	Definitions of state, remember, mutableStateOf, hoisting, recomposition, flow diagram, strategy table, reflection
Task 2	Input, Validation and UI Feedback Plan	Expense Splitter feature, input/state listing, validation rules, UI feedback wireframe, reflection
Task 3	Comprehensive Feature Design and Implementation Strategy	Bill Split and Tip Tracker, state model, pseudo-code, test cases, scaling plan, reflection

Task 1 - State Concepts Deep Dive and Visual Mapping

1. Key Term Definitions

State

State is any value in the app that can change over time and whose change should cause the UI to update. In Jetpack Compose, UI is a function of state.

```
var isDarkMode by remember { mutableStateOf(false) }
```

When `isDarkMode` changes -> Compose redraws the UI.

remember

`remember { }` stores a value across recompositions so it is not recreated every time.

```
var count by remember { mutableStateOf(0) }
```

mutableStateOf()

Creates an observable state holder. When its value changes, Compose recomposes dependent composables.

```
val text = mutableStateOf("")
text.value = "Hello"
```

State Hoisting

State hoisting means moving state from a child composable to its parent or ViewModel.

```
@Composable
fun NameField(value: String, onValueChange: (String) -> Unit)

var name by remember { mutableStateOf("") }
NameField(value = name, onValueChange = { name = it })
```

Recomposition

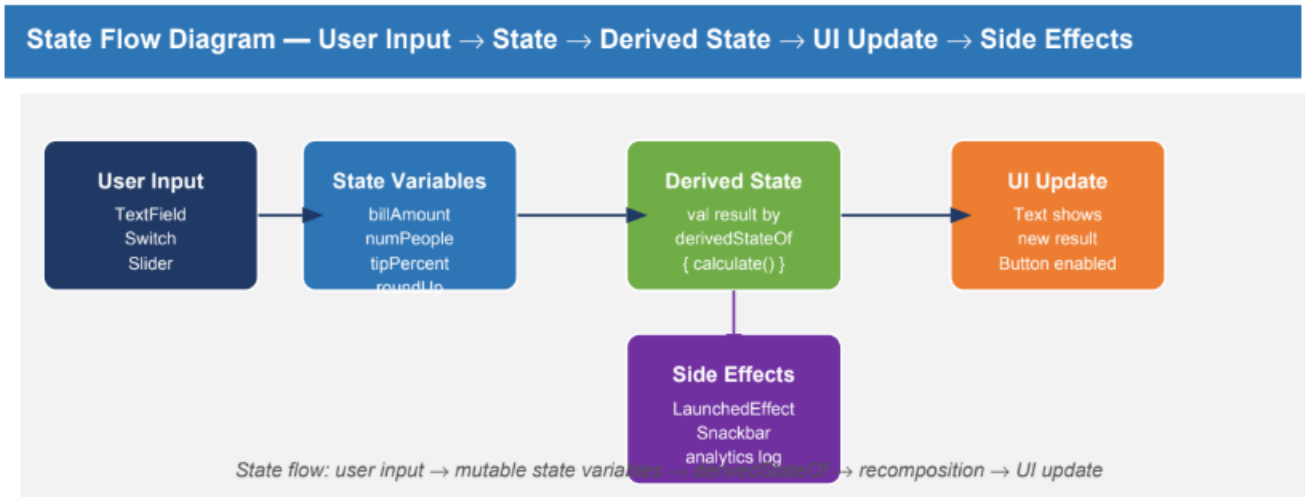
Recomposition is the re-execution of composables when observed state changes.

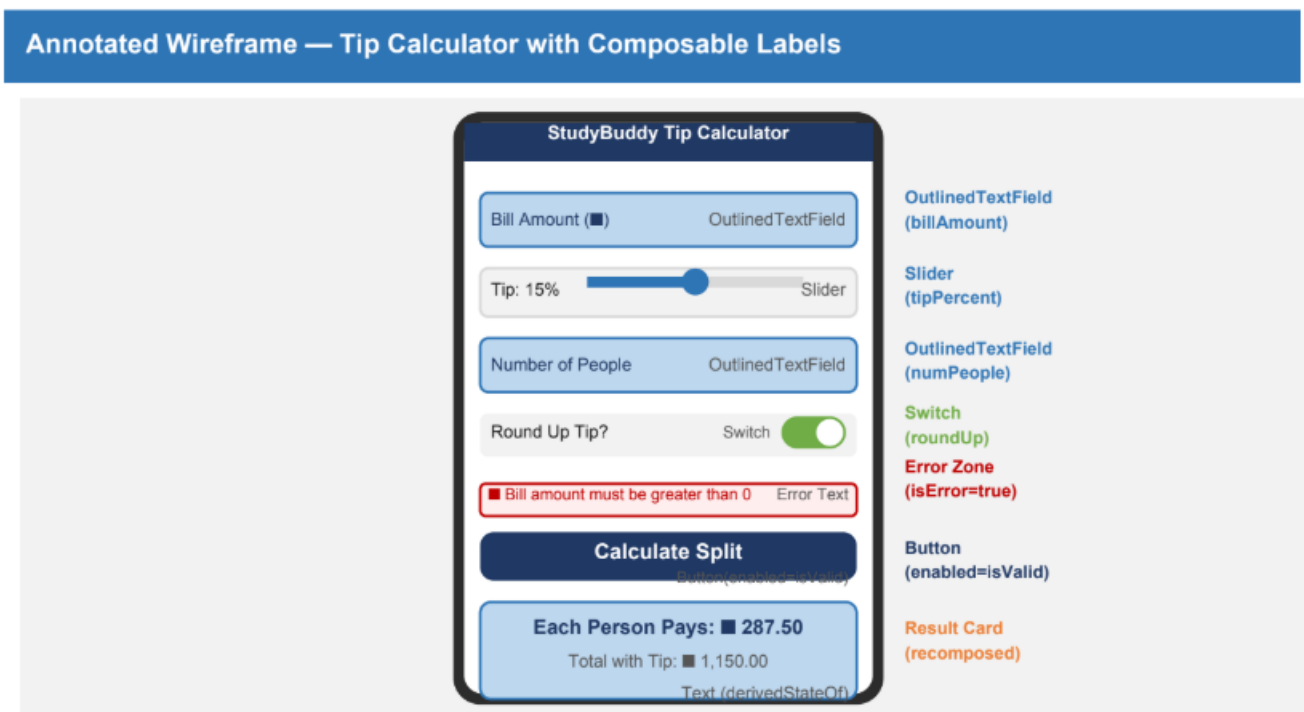
```
Text("Count: $count")
Image(...)
```

Only composables that read changed state are recomposed.

2. Interaction Scenario - Tip Calculator State Flow

User Input -> Mutable State -> derivedStateOf -> Recomposition -> UI Update





Each UI element labelled with its composable type and state variable. The error zone appears only when isError=true. Result card recomposes on every state change.

3. State Handling Strategy Table

UI Component	State Variable	Stateful or Stateless	Defined Where	Notes
Bill TextField	billAmountInput	Stateless	ViewModel	Error when value <= 0
Slider	tipPercent	Stateless	ViewModel	Range 0f to 30f
People TextField	numPeopleInput	Stateless	ViewModel	Must be >= 1
Switch	roundUp	Stateless	ViewModel	Recalculates result
Result Text	result	Stateless	derivedStateOf	Auto re-composes

4. Reflection

State hoisting ensures: - Single source of truth
- Better testing
- Easier maintenance
- Correct configuration handling

If state is kept locally using `remember`, rotation may reset UI values and cause inconsistency. Hoisting fixes this issue.

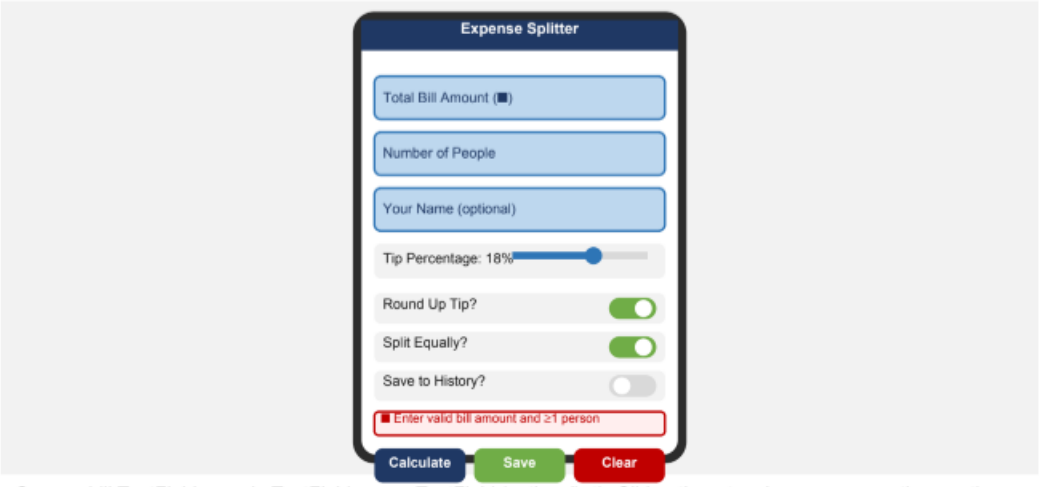
Task 2 - Input, Validation and UI Feedback Plan

Feature: StudyBuddy Expense Splitter

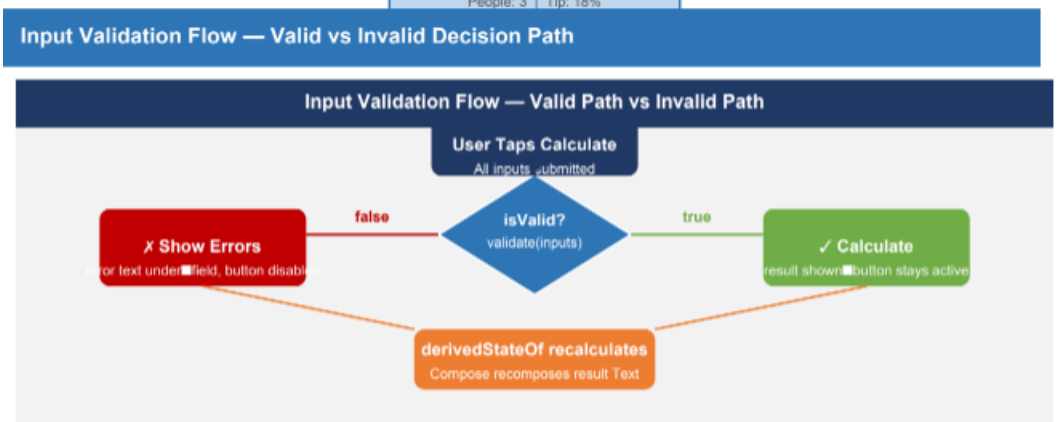
Users can: - Enter bill amount
- Enter number of people
- Adjust tip percentage
- Toggle round up
- Save result

1. Input and State Variables

State Variable	Type	Validation
billAmountInput	String	Must be > 0
numPeopleInput	String	Must be >= 1
tipPercent	Float	0f to 30f
roundUp	Boolean	No validation
isFormValid	Boolean	Derived



Screen: bill TextField, people TextField, name TextField (optional), tip Slider, three toggles, error zone, three action buttons (Calculate/Save/Clear), result card.



Central diamond: isValid check on Calculate tap. True path → derivedStateOf recalculates. False path → error states shown, button stays disabled.

2. Validation Logic

```
val isBillValid by derivedStateOf {
    billAmountInput.isNotBlank() &&
    billAmountInput.toDoubleOrNull()?.let { it > 0 } == true
}

val isNumPeopleValid by derivedStateOf {
    numPeopleInput.isNotBlank() &&
    numPeopleInput.toIntOrNull()?.let { it >= 1 } == true
}

val isFormValid by derivedStateOf {
    isBillValid && isNumPeopleValid
}
```

3. UI Feedback States

Valid State: - Normal border - Calculate enabled

Invalid State: - Red border - Error message - Button disabled

Empty State: - Placeholder visible - No errors shown

4. Reflection

Validation prevents: - Division by zero
- Crashes
- Incorrect calculations

Using derivedStateOf ensures reactive and efficient updates.

Task 3 - Comprehensive Feature Design and Implementation Strategy

Feature: Bill Split and Tip Tracker

Additional Features: - Save history
- Delete entries
- Clear history
- Preserve state on rotation

Left: calculator with inputs, toggles, Calculate/Save/Clear buttons, result card. Right: scrollable LazyColumn history list with per-entry delete and Clear History button.

1. State Model

Variable	Type	Initial
billAmountInput	String	“”
numPeopleInput	String	“”
tipPercent	Float	15f
roundUp	Boolean	false
isCalculated	Boolean	false
historyList	List	emptyList()

2. ViewModel Pseudo Code

```
class BillSplitViewModel : ViewModel() {

    var billAmountInput by mutableStateOf("")
    var numPeopleInput by mutableStateOf("")
    var tipPercent by mutableStateOf(15f)
    var roundUp by mutableStateOf(false)
    var isCalculated by mutableStateOf(false)
```

```
    val isBillValid by derivedStateOf {
        billAmountInput.isNotBlank() &&
        billAmountInput.toDoubleOrNull()?.let { it > 0 } == true
    }

    val isNumPeopleValid by derivedStateOf {
        numPeopleInput.isNotBlank() &&
        numPeopleInput.toIntOrNull()?.let { it >= 1 } == true
    }

    val isFormValid by derivedStateOf {
        isBillValid && isNumPeopleValid
    }

    val result by derivedStateOf {
        if (!isFormValid) return@derivedStateOf null
        val bill = billAmountInput.toDouble()
        val people = numPeopleInput.toInt()
        val tipAmount = bill * tipPercent / 100
        val total = if (roundUp) kotlin.math.ceil(bill + tipAmount) else bill + tipAmount
        total / people
    }
}
```

3. Test Cases

Test Case	Expected Outcome
Valid input	Correct split
Bill empty	Error shown
People = 0	Prevent calculation
Large numbers	No crash
Rotation	State preserved

Final Reflection

This assessment strengthened understanding of:

- State as single source of truth
- Proper validation
- derivedStateOf for efficiency
- ViewModel for business logic

Conclusion

Proper state management and validation create reliable, scalable, and maintainable Compose applications.